DATA 37711 — Homework 3

Zhengxin Lu

Problem 1

(a) i. $Y \in \{0, 1, 2, \dots\}$ is a count, and the Poisson distribution is the standard model for counts on the nonnegative integers. One caveat: it forces $\mathbb{E}[Y \mid X] = \mathrm{Var}(Y \mid X)$, so it is less appropriate for overdispersed or zero-inflated data.

(a) ii. The Poisson rate must be positive, but the tree ensemble outputs $f(X) \in \mathbb{R}$. The map $\exp : \mathbb{R} \to (0, \infty)$ guarantees a valid rate $\mu(x) = e^{f(x)}$. Moreover, log is the canonical link of the Poisson family, so the negative log-likelihood is convex in f.

(b) The loss is the negative log-likelihood:

$$L(f(x), Y) = -\log \frac{e^{-e^{f(x)}} e^{Y f(x)}}{Y!} = e^{f(x)} - Y f(x) + \log Y!.$$

The term $\log Y!$ does not depend on f, so we minimize $\hat{R}(f) = \frac{1}{n} \sum_i \big(e^{f(x_i)} - Y_i f(x_i)\big)$.

(c) $\dfrac{dL}{df(x)} = e^{f(x)} - Y.$

(d) Initialize $f_0 \equiv \log \bar{Y}$, the best constant (assuming $\bar{Y} > 0$). For $m = 1, 2, \dots$:

1. Compute the pseudo-residuals (negative gradients) $r_i = Y_i - e^{f_{m-1}(x_i)}$.
2. Fit a regression tree to them by least squares: $h_m = \arg\min_h \sum_i (r_i - h(x_i))^2$.
3. Choose the step size by line search: $\lambda_m = \arg\min_\lambda \sum_i L\big(f_{m-1}(x_i) + \lambda h_m(x_i), Y_i\big)$.
4. Update $f_m = f_{m-1} + \nu \lambda_m h_m$, with learning rate $0 < \nu \leq 1$.

Problem 2

(a) The k hyperparameter settings are fixed in advance. Train each on the n training points to obtain $\hat{h}_1, \dots, \hat{h}_k$. Let $\hat{R}_j = \frac{1}{l} \sum_{i=1}^{l} \ell(\hat{h}_j(x'_i), y'_i)$ be the error on an independent test set of size l, and $R_j = \mathbb{E}[\ell(\hat{h}_j(X), Y) \mid \text{train}]$ the true risk. Choose $\hat{j} = \arg\min_j \hat{R}_j$. Assume $0 \leq \ell \leq B$.

Proposition. *With probability at least* $1 - \delta$,

$$\max_{1 \leq j \leq k} |\hat{R}_j - R_j| \leq B \sqrt{\frac{\log(2k/\delta)}{2l}} =: \epsilon.$$

In particular, $|\hat{R}_{\hat{j}} - R_{\hat{j}}| \leq \epsilon$ *and* $R_{\hat{j}} \leq \min_j R_j + 2\epsilon$.

Proof. Condition on the training set; then each $\hat{h}_j$ is a fixed function, and the test points are i.i.d. and independent of it. For a fixed j, $\hat{R}_j$ is an average of l i.i.d. $[0, B]$-valued variables with mean R_j. By Hoeffding's inequality, $\Pr(|\hat{R}_j - R_j| > \epsilon) \leq 2e^{-2l\epsilon^2/B^2}$. A union bound over the k candidates gives $\Pr(\max_j |\hat{R}_j - R_j| > \epsilon) \leq 2ke^{-2l\epsilon^2/B^2}$. Setting this equal to δ and solving gives the stated ϵ.

Since the bound holds conditionally on every training set, it also holds unconditionally. Because $\hat{j}$ is one of the k indices, the bound covers it. For the second claim, let $j^* = \arg\min_j R_j$. Then

$$R_{\hat{j}} \le \hat{R}_{\hat{j}} + \epsilon \le \hat{R}_{j^*} + \epsilon \le R_{j^*} + 2\epsilon. \qquad \square$$

The bound has no explicit dependence on n, because it concerns only the evaluation of already-trained models; n affects how small the R_j themselves are. Boundedness is required. For the squared loss in (c), it follows from the problem setup: quality lies in $\{1, \dots, 10\}$, and both RF and k-NN predictions are averages of training labels, so they also lie in $[1, 10]$. Hence $B = 81$.

(b) The selected test error $\hat{R}_{\hat{j}} = \min_j \hat{R}_j$ is optimistically biased. Conditional on training,

$$\mathbb{E}\big[\min_j \hat{R}_j\big] \le \min_j \mathbb{E}[\hat{R}_j] = \min_j R_j \le \mathbb{E}[R_{\hat{j}}].$$

We pick the candidate that happened to do well on these l points. By (a), this effect is at most of order $\sqrt{\log k / l}$, and it grows with k. When k is large relative to l, the selection fits noise in the test set, and the reported error no longer estimates the true risk. To avoid this, tune on a validation set and evaluate on the test set only once.

(c) I used the provided `winequality-white.csv` (4,898 rows). A random split (seed 0) gives 3,428 training, 490 validation, and 980 test points. For each model class, I trained every configuration on the training set, picked the one with the lowest validation MSE, and then evaluated it once on the test set.

- **RandomForestRegressor** (600 configurations): number of trees $\in \{50, 100\}$, max depth $\in \{\text{None}, 4, 8, 12, 16, 24\}$, min samples per leaf $\in \{1, 2, 4, 8, 16\}$, max features $\in \{.2, .33, .5, .75, 1\}$, bootstrap $\in \{T, F\}$.
- **KNeighborsRegressor** (600 configurations; features standardized using training-set statistics): neighbors $\in \{1, \dots, 100\}$, weights $\in \{\text{uniform, distance}\}$, Minkowski $p \in \{1, 1.5, 2\}$.

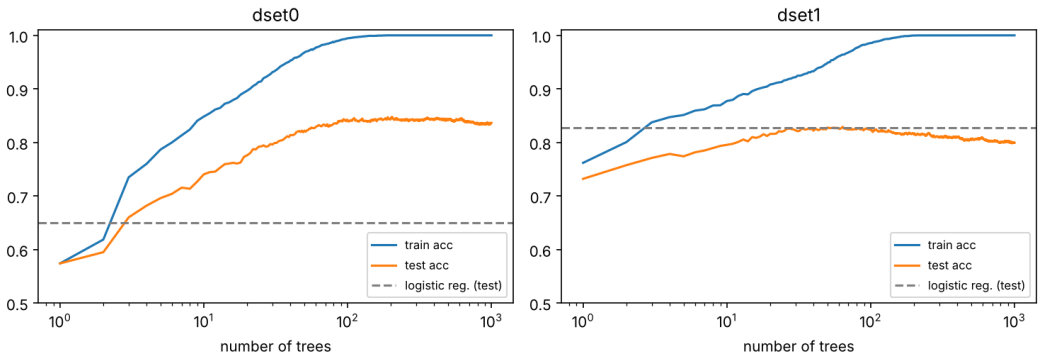
Model	Selected parameters	Val. MSE	Test MSE
RF	50 trees, depth 16, max_features .2, leaf 1, no bootstrap	0.362	0.439
KNN	$k = 16$, distance weights, $p = 1$	0.400	0.462

For both models, the validation MSE of the selected configuration is lower than its test MSE. This is consistent with the selection-induced optimism described in (b): the validation MSE of the chosen model is a minimum over 600 noisy estimates. The observed gap also reflects sampling variation between the two sets. In expectation, $\mathbb{E}[\hat{R}_{\text{val}}(\hat{h}_{\hat{j}})] \le \mathbb{E}[R(\hat{h}_{\hat{j}})] = \mathbb{E}[\hat{R}_{\text{test}}(\hat{h}_{\hat{j}})]$. The test MSE is therefore an unbiased estimate of the selected model's risk, because the test set played no role in the selection.

Problem 3

I used `GradientBoostingClassifier` with depth-3 trees and learning rate 0.1. All numbers are averages over 5-fold stratified CV repeated 4 times; standard deviations of test accuracy across folds are in parentheses.

	dset0			dset1		
Trees	train acc	test acc	test log-loss	train acc	test acc	test log-loss
1	.574	.574 (.005)	.663	.762	.732 (.043)	.648
10	.848	.740 (.045)	.571	.877	.795 (.035)	.477
100	.995	.842 (.025)	.376	.986	.824 (.031)	.393
300	1.000	.842 (.023)	.394	1.000	.809 (.035)	.502
1000	1.000	.836 (.024)	.684	1.000	.799 (.033)	.950
Logistic reg.		.650			.828	



(a) dset0. Test accuracy rises steadily up to about 100–300 trees ($\approx .84$). From there through 1000 trees, test accuracy remains approximately stable ($.842 \to .836$), even though training accuracy reaches 100%. Test log-loss, however, rises after about 100 trees, because the predicted probabilities become overconfident.

(b) dset1. Test accuracy peaks earlier, around 100 trees (.824), and then declines to .799 at 1000 trees, as the model interpolates the training data. Test log-loss worsens sharply ($.39 \to .95$). At the evaluated tree counts, boosting does not outperform logistic regression (.828). Here the number of trees acts as a regularization parameter, and early stopping helps.

(c) The behaviour differs mainly in test accuracy; on both datasets, log-loss eventually overfits. A plausible explanation is that the two datasets differ in how complex the signal is relative to the noise:

- *dset0* appears to have a nonlinear signal involving interactions. Logistic regression reaches only .65, while boosting reaches .84. The additional trees keep learning real structure, so accuracy holds up even after the model fits the training data perfectly.
- *dset1* appears to have a simpler, near-linear decision boundary. Logistic regression (.83) matches boosting, and x_1 alone is strongly predictive. Once the boundary has been learned, later trees are likely fitting label noise or spurious feature patterns, so test accuracy declines.

This interpretation is consistent with the observations, but it is not a proof of the data-generating mechanism.